PYTHON DICTIONARY

Complete Teaching Notes: Basics to Practical Programs

For classroom teaching and student revision

Topics include dictionary fundamentals, syntax, keys and values, indexing versus keys, methods, loops, range(), nested dictionaries, comprehension, common mistakes, and practice questions.

Python 3 • Beginner-friendly examples • Outputs included

1. What is a dictionary?

A dictionary is a built-in Python mapping type that stores data as key-value pairs. A key is used to look up its associated value. Dictionaries are mutable and preserve insertion order in modern Python (Python 3.7+). Keys must be unique and hashable; values can be duplicated and can hold different data types.

```
student = {
    "roll_no": 101,
    "name": "Shubham",
    "course": "Python",
    "marks": 85
}

print(student)
```

Output: {'roll_no': 101, 'name': 'Shubham', 'course': 'Python', 'marks': 85}

2. Syntax and rules

```
variable_name = {
    key1: value1,
    key2: value2
}
```

• Use curly braces {} for a dictionary literal.
• A colon separates each key from its value; commas separate pairs.
• Keys must be unique. If a key is repeated, the last assigned value is retained.
• Keys must be hashable. Common choices are strings, integers, and tuples containing only hashable elements. Lists and dictionaries cannot be keys.
• Values can be any data type, including lists, tuples, sets, and other dictionaries.
• An empty dictionary is {}. Use set() to create an empty set.

Creating dictionaries

```
student = {"name": "Rahul", "age": 22, "marks": 85}
employee = {"id": 101, "name": "Amit", "salary": 35000}
empty = {}

print(type(empty))
```

Output:

3. Accessing values

Square-bracket access

```
student = {"name": "Rahul", "age": 22, "marks": 85}
print(student["name"])
print(student["marks"])
```

Output: Rahul\n85. A missing key raises KeyError.

get() access

```
student = {"name": "Rahul", "age": 22}
print(student.get("name"))
print(student.get("city"))
print(student.get("city", "Not Available"))
```

Output: Rahul\nNone\nNot Available. get() does not raise KeyError for a missing key.

4. Indexing and slicing

Dictionaries do not support positional indexing or slicing. student[0] means “look up key 0,” not “get the first entry.” It works only if 0 is an actual key.

```
data = {"name": "Rahul", "age": 22}
# data[0]       -> KeyError (key 0 is absent)
# data[0:2]     -> TypeError (dictionary slicing is unsupported)

data2 = {0: "Rahul", 1: "Amit"}
print(data2[0])  # Rahul
```

If you need positional access, convert keys, values, or items to a list first.

```
student = {"name": "Rahul", "age": 22, "marks": 85}
print(list(student.items())[:2])
# [('name', 'Rahul'), ('age', 22)]
```

5. Adding, updating, and deleting

Add or update with assignment

```
student = {"name": "Rahul", "marks": 75}
student["city"] = "Pune"     # add
student["marks"] = 90        # update
print(student)
```

Output: {'name': 'Rahul', 'marks': 90, 'city': 'Pune'}

update()

```
student = {"name": "Rahul", "marks": 75}
student.update({"marks": 90, "city": "Pune"})
print(student)
```

Output: {'name': 'Rahul', 'marks': 90, 'city': 'Pune'}

del

```
student = {"name": "Rahul", "age": 22}
del student["age"]
print(student)
```

Output: {'name': 'Rahul'}

6. All 11 standard dictionary methods

These are the 11 standard public dictionary methods. Built-in functions such as len(), type(), sorted(), and sum() are not dictionary methods.

Method	Purpose	Changes dictionary?
get(key, default)	Get value safely; missing key returns None or default	No
keys()	View of all keys	No
values()	View of all values	No
items()	View of (key, value) pairs	No
update(other)	Add or update pairs	Yes
pop(key, default)	Remove key and return its value; default optional	Yes
popitem()	Remove and return last inserted pair	Yes
clear()	Remove all pairs	Yes
copy()	Create a shallow copy	Creates new dict
setdefault(key, default)	Return existing value or insert default	Only if key missing
dict.fromkeys(keys, value)	Create a new dictionary from keys with same value	Creates new dict

1. get()

```
student = {"name": "Rahul"}
print(student.get("city", "Unknown"))
```

Output: Unknown

2. keys()

```
student = {"name": "Rahul", "age": 22}
print(list(student.keys()))
```

Output: ['name', 'age']

3. values()

```
marks = {"Python": 85, "Java": 90, "SQL": 80}
print(list(marks.values()))
print(sum(marks.values()))
```

Output: [85, 90, 80]\n265

4. items()

```
student = {"name": "Rahul", "age": 22}
for key, value in student.items():
    print(key, "=", value)
```

Output: name = Rahul\nage = 22

5. update()

```
student = {"name": "Rahul", "marks": 75}
student.update({"marks": 95, "city": "Pune"})
print(student)
```

Output: {'name': 'Rahul', 'marks': 95, 'city': 'Pune'}

6. pop()

```
student = {"name": "Rahul", "age": 22}
age = student.pop("age")
print(age)
print(student)
```

Output: 22\n{'name': 'Rahul'}. A missing key raises KeyError unless a default is supplied.

7. popitem()

```
student = {"name": "Rahul", "age": 22}
print(student.popitem())
print(student)
```

Output: ('age', 22)\n{'name': 'Rahul'}. Empty dictionaries raise KeyError.

8. clear()

```
student = {"name": "Rahul", "age": 22}
student.clear()
print(student)
```

Output: {}

9. copy()

```
original = {"name": "Rahul", "marks": 85}
copied = original.copy()
copied["marks"] = 95
print(original)
print(copied)
```

Output: {'name': 'Rahul', 'marks': 85}\n{'name': 'Rahul', 'marks': 95}. copy() is shallow; nested mutable values are shared.

10. setdefault()

```
student = {"name": "Rahul"}
print(student.setdefault("city", "Pune"))
print(student.setdefault("name", "Amit"))
print(student)
```

Output: Pune\nRahul\n{'name': 'Rahul', 'city': 'Pune'}

11. dict.fromkeys()

```
subjects = ["Java", "Python", "SQL"]
marks = dict.fromkeys(subjects, 0)
print(marks)
```

Output: {'Java': 0, 'Python': 0, 'SQL': 0}. Avoid a mutable default such as [] if each key needs a separate list.

7. Traversing a dictionary

Traverse keys

```
student = {"name": "Rahul", "age": 22, "marks": 85}
for key in student:
    print(key)
```

Prints name, age, marks on separate lines.

Traverse values

```
for value in student.values():
    print(value)
```

Prints Rahul, 22, 85 on separate lines.

Traverse key-value pairs

```
for key, value in student.items():
    print(key, ":", value)
```

Prints each key alongside its value.

Filter values using a condition

```
marks = {"Rahul": 85, "Amit": 60, "Priya": 92, "Neha": 68}
for name, score in marks.items():
    if score > 70:
        print(name, score)
```

Output: Rahul 85\nPriya 92

8. Can range() be used with a dictionary?

Yes, range() can control a loop count, but it does not make a dictionary position-indexed. Convert keys, values, or items to a list before using positional indexes.

Using range() to traverse keys

```
student = {"name": "Rahul", "age": 22, "marks": 85}
keys = list(student.keys())
for i in range(len(student)):
    print(keys[i])
```

Output: name\nage\nmarks

Using range() to traverse pairs

```
student = {"name": "Rahul", "age": 22, "marks": 85}
items = list(student.items())
for i in range(len(items)):
    print(items[i][0], ":", items[i][1])
```

Output: name : Rahul\nage : 22\nmarks : 85

Incorrect for ordinary string-keyed dictionaries: for i in range(len(student)): print(student[i]). This raises KeyError because i is not one of the dictionary keys. If the dictionary has integer keys 0, 1, 2, then student[i] can work because those are actual keys.

9. Membership, length, and built-in functions

in and not in

```
student = {"name": "Rahul", "marks": 85}
print("name" in student)          # True
print("age" in student)           # False
print(85 in student)              # False: checks keys
print(85 in student.values())     # True
```

len()

```
student = {"name": "Rahul", "age": 22, "marks": 85}
print(len(student))  # 3
```

sorted() and sum()

```
marks = {"Python": 85, "Java": 90, "SQL": 80}
print(sorted(marks))          # sorted list of keys
print(sum(marks.values()))    # 255
```

10. Different data types and nested dictionaries

Values can be different data types

```
student = {
    "name": "Rahul",
    "age": 22,
    "skills": ["Java", "Python", "SQL"],
    "address": ("Pune", "Maharashtra"),
    "marks": {"Python": 85, "Java": 90}
}
print(student["skills"])
print(student["address"])
print(student["marks"]["Python"])
```

Output: ['Java', 'Python', 'SQL']\n('Pune', 'Maharashtra')\n85

Nested dictionary

```
students = {
    101: {"name": "Rahul", "marks": 85},
    102: {"name": "Priya", "marks": 92}
}
print(students[101]["name"])
print(students[102]["marks"])
```

Output: Rahul\n92

11. Duplicate keys and values

```
data = {"name": "Rahul", "marks": 70, "marks": 90}
print(data)
# {'name': 'Rahul', 'marks': 90}

scores = {"student1": 90, "student2": 90, "student3": 75}
# Duplicate values are allowed.
```

12. Dictionary comprehension

```
squares = {number: number ** 2 for number in range(1, 6)}
print(squares)
# {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

numbers = [1, 2, 3, 4, 5, 6]
even_squares = {
    number: number ** 2
    for number in numbers
    if number % 2 == 0
}
print(even_squares)
# {2: 4, 4: 16, 6: 36}
```

Use comprehension after students understand dictionaries, loops, and conditions.

13. Practical program: student report

```
students = {
    101: {"name": "Rahul", "marks": 85},
    102: {"name": "Priya", "marks": 92},
    103: {"name": "Amit", "marks": 60}
}

for roll_no, details in students.items():
    print("Roll Number:", roll_no)
    print("Name:", details["name"])
    print("Marks:", details["marks"])
    if details["marks"] >= 70:
        print("Result: Good")
    else:
        print("Result: Needs Improvement")
    print("----------------")
```

14. Common mistakes

- Using student[0] expecting the first entry; dictionary lookup uses keys.
- Using slicing such as student[0:2]; dictionaries do not support slicing.
- Assuming in checks values; by default, it checks keys.
- Using a list or dictionary as a key; these are unhashable.
- Forgetting that assigning to an existing key replaces its value.
- Assuming copy() deeply copies nested mutable objects.
- Using dict.fromkeys(keys, []) when each key needs its own independent list.
- Confusing {} (empty dictionary) with set() (empty set).

15. Practice questions

1. Create a dictionary containing your name, age, and city.
2. Print only the name from a student dictionary.
3. Add an email key to an existing dictionary.
4. Update an employee's salary.
5. Remove the age key from a dictionary.
6. Print all keys and values using a for loop.
7. Check whether the key 'course' exists.
8. Count the number of key-value pairs.

9. Create a nested dictionary for three students.

10. Create a dictionary of numbers and their squares from 1 to 10.

11. Print only students who scored more than 75.

12. Count the frequency of each character in a string using a dictionary.

13. Use range() to print dictionary entries by position after converting items to a list.

14. Use fromkeys() to initialize five subject marks to zero.

15. Copy a dictionary and demonstrate that changing a top-level value in the copy does not change the original.

16. Quick revision

Question	Answer
How are dictionary entries stored?	As key-value pairs
Are dictionaries mutable?	Yes
Can keys repeat?	Keys must be unique; later assignment replaces earlier value
Can values repeat?	Yes
Does a dictionary support positional indexing?	No; bracket access looks up a key
Does a dictionary support slicing?	No
Can dictionaries be traversed?	Yes, using for or while loops
Can range() be used?	Yes, for loop counts; convert items to a list for positional processing
How many standard public dictionary methods are taught here?	11